GATE_ACCESS_DBSETUP · INFRASTRUCTURE RUNBOOK

Hosting the Gate Access Database on AWS

Amazon Aurora (managed) and self-managed MySQL/PostgreSQL on EC2 — production setup, from environment configuration through hardening.

Covers: **PostgreSQL** (current, `app/`) and **MySQL** (new, `app_mysql/`)

Target environment: **Production**

Prepared for: gate_access_dbsetup

> **Before you run anything here:** every command below creates or changes billed AWS resources. Review instance sizes and Multi-AZ choices against your budget before executing, and confirm you're in the intended AWS account/region (`aws sts get-caller-identity`, `aws configure get region`).

0. Decision summary

1. Prerequisites

2. Environment & secrets changes

3. Path A — Amazon Aurora

4. Path B — Self-managed on EC2

5. Migrating existing data

6. Production hardening checklist

7. Rotate the credentials already exposed in this repo

Appendix A. MySQL from Lambda — driver, Aurora, IAM auth

0. Decision summary

Both engines are the same choice structurally — Aurora vs. EC2 is about who operates the database, not which schema it holds. `app/` (Postgres) and `app_mysql/` (MySQL) each work unchanged against either hosting path.

	Amazon Aurora	**MySQL/Postgres on EC2**
Ops burden	AWS patches, backs up, and fails over automatically	You patch the OS and the database engine, write your own backup jobs
HA	Storage replicated across 3 AZs by design; add a reader instance for automatic failover	Manual — a second instance + replication you configure and monitor yourself
Cost floor	No free tier; smallest sensible prod instance (db.t4g.medium) is roughly $80–120/mo before storage/IO	A prod-sized t3.small/medium runs roughly $15–30/mo compute; you add EBS + snapshot cost
Best for	Production where downtime and manual recovery are costly	Tight budgets, or workloads that already assume hands-on server management (matches how your current Postgres box at `54.246.247.31` is run today)

> Recommendation for a production gate-access system (physical security, needs to survive an AZ outage): **Aurora**. Use the EC2 path if cost is the binding constraint and you're comfortable owning failover/patching.

1. Prerequisites

1. Authenticate the AWS CLI in this environment (currently **not** authenticated — `aws sts get-caller-identity` returned `InvalidClientTokenId`):

```
aws configure
# AWS Access Key ID / Secret Access Key / default region (e.g. us-east-1) / output format: json

# verify:
aws sts get-caller-identity
```

2. Confirm you have a VPC with at least two private subnets (different AZs) for the database, and a public subnet for anything (bastion/NAT) that needs internet egress. If you're using the AWS-managed default VPC, this already exists — fetch the IDs:

```
aws ec2 describe-vpcs --query 'Vpcs[].VpcId'
aws ec2 describe-subnets --filters "Name=vpc-id,Values=<VPC_ID>" \
  --query 'Subnets[].{Id:SubnetId,AZ:AvailabilityZone,Public:MapPublicIpOnLaunch}'
```

Set these as shell variables for the rest of this guide:

```
export VPC_ID=vpc-xxxxxxxx
export SUBNET_A=subnet-xxxxxxxx   # private, AZ a
export SUBNET_B=subnet-xxxxxxxx   # private, AZ b
export REGION=us-east-1
```

3. Have the app server's identity ready — either an existing EC2 security group ID (the app tier) or its IP range — so the database security group can allow *only* that traffic, never `0.0.0.0/0`.

2. Environment & secrets changes

Both `app/db/database.py` and `app_mysql/db/database.py` already read from environment variables with a local fallback:

```
# app/db/database.py
DATABASE_URL = os.environ.get("DATABASE_URL", "postgresql://...")   # currently hardcoded,
see §7

# app_mysql/db/database.py
DATABASE_URL = os.environ.get("MYSQL_DATABASE_URL", "mysql+pymysql://...")
```

2.1 For production, drop the hardcoded fallback entirely

A fallback with a real password baked into source means the password ships every time the repo is cloned or read. Change both files to require the env var:

```
import os
DATABASE_URL = os.environ["DATABASE_URL"]   # raises KeyError if unset — fail loud, not
silent
```

2.2 `.env` — local/dev only

Keep `.env` for local development, never for the production host:

```
# .env  (local dev only — do not deploy this file)
DATABASE_URL=postgresql+psycopg2://gate_app:<password>@<pg-host>:5432/gate_access?
sslmode=require
MYSQL_DATABASE_URL=mysql+pymysql://gate_app:<password>@<mysql-host>:3306/gate_access
OPENAI_API_KEY=<rotated key — see §7>
```

2.3 Production — use AWS Secrets Manager instead of a file

Store the connection string as a secret and fetch it at process start, so the credential never touches disk on the app host:

```
# one-time, per environment
aws secretsmanager create-secret \
  --name gate-access/prod/database-url \
  --secret-string "postgresql+psycopg2://gate_app:<password>@<pg-host>:5432/gate_access?
sslmode=require"

aws secretsmanager create-secret \
  --name gate-access/prod/mysql-database-url \
  --secret-string "mysql+pymysql://gate_app:<password>@<mysql-host>:3306/gate_access"
```

```
# app/db/database.py — fetch instead of os.environ, e.g.:
import boto3, json, os

def _load_secret(name: str) -> str:
    client = boto3.client("secretsmanager", region_name=os.environ["AWS_REGION"])
    return client.get_secret_value(SecretId=name)["SecretString"]

DATABASE_URL = os.environ.get("DATABASE_URL") or _load_secret("gate-access/prod/database-
url")
```

Grant the EC2 instance (or its instance role) `secretsmanager:GetSecretValue` on that secret ARN only — via an IAM instance profile, not static access keys on the box.

3. Path A — Amazon Aurora

Steps are nearly identical for both engines; differences are called out inline. Run the `POSTGRES` block for `app/`, the `MYSQL` block for `app_mysql/` — you can run both if you want to keep them side by side.

3.1 Subnet group and security group (shared by both engines)

```
aws rds create-db-subnet-group \
  --db-subnet-group-name gate-access-db-subnets \
  --db-subnet-group-description "Private subnets for gate_access Aurora" \
  --subnet-ids "$SUBNET_A" "$SUBNET_B"

aws ec2 create-security-group \
  --group-name gate-access-db-sg \
  --description "DB access for gate_access" --vpc-id "$VPC_ID" \
  --query 'GroupId' --output text
export DB_SG=sg-xxxxxxxx   # from the output above

# allow only the app tier's security group in — never 0.0.0.0/0 for a prod DB
aws ec2 authorize-security-group-ingress \
  --group-id "$DB_SG" --protocol tcp --port 5432 \
  --source-group "$APP_SG"     # Postgres port; use 3306 for MySQL
```

3.2 `POSTGRES` Create the Aurora PostgreSQL cluster

```
# find a currently supported engine version
aws rds describe-db-engine-versions --engine aurora-postgresql \
  --query 'DBEngineVersions[].EngineVersion' --output table

aws rds create-db-cluster \
  --db-cluster-identifier gate-access-pg \
  --engine aurora-postgresql --engine-version <version-from-above> \
  --master-username gate_app --master-user-password '<strong-password>' \
  --db-subnet-group-name gate-access-db-subnets \
  --vpc-security-group-ids "$DB_SG" \
  --database-name gate_access \
  --backup-retention-period 7 \
  --no-publicly-accessible \
  --storage-encrypted

# writer instance
aws rds create-db-instance \
  --db-instance-identifier gate-access-pg-writer \
  --db-cluster-identifier gate-access-pg \
  --engine aurora-postgresql \
  --db-instance-class db.t4g.medium

# reader instance in a second AZ — gives you automatic failover
aws rds create-db-instance \
  --db-instance-identifier gate-access-pg-reader \
  --db-cluster-identifier gate-access-pg \
  --engine aurora-postgresql \
  --db-instance-class db.t4g.medium

aws rds wait db-instance-available --db-instance-identifier gate-access-pg-writer
aws rds describe-db-clusters --db-cluster-identifier gate-access-pg \
  --query 'DBClusters[0].Endpoint'   # write endpoint
```

3.3 MYSQL Create the Aurora MySQL cluster

```
aws rds describe-db-engine-versions --engine aurora-mysql \
  --query 'DBEngineVersions[].EngineVersion' --output table

aws rds create-db-cluster \
  --db-cluster-identifier gate-access-mysql \
  --engine aurora-mysql --engine-version <version-from-above> \
  --master-username gate_app --master-user-password '<strong-password>' \
  --db-subnet-group-name gate-access-db-subnets \
  --vpc-security-group-ids "$DB_SG" \
  --database-name gate_access \
  --backup-retention-period 7 \
  --no-publicly-accessible \
  --storage-encrypted

aws rds create-db-instance \
  --db-instance-identifier gate-access-mysql-writer \
  --db-cluster-identifier gate-access-mysql \
  --engine aurora-mysql \
  --db-instance-class db.t4g.medium

aws rds create-db-instance \
  --db-instance-identifier gate-access-mysql-reader \
  --db-cluster-identifier gate-access-mysql \
  --engine aurora-mysql \
  --db-instance-class db.t4g.medium

aws rds wait db-instance-available --db-instance-identifier gate-access-mysql-writer
aws rds describe-db-clusters --db-cluster-identifier gate-access-mysql \
  --query 'DBClusters[0].Endpoint'
```

3.4 Point the app at it and create the schema

```
# Postgres — from a host that can reach the private subnet (app server, bastion, or SSM)
export DATABASE_URL="postgresql+psycopg2://gate_app:<password>@<writer-
endpoint>:5432/gate_access?sslmode=require"
cd gate_access_dbsetup
pipenv run alembic upgrade head

# MySQL — no migration history yet (see §4 note in the prior conversation), so create
tables directly:
export MYSQL_DATABASE_URL="mysql+pymysql://gate_app:<password>@<writer-
endpoint>:3306/gate_access"
python3 -c "
from app_mysql.db.database import Base, engine
from app_mysql.db import models
Base.metadata.create_all(engine)
"
```

> Aurora endpoints resolve to the writer automatically and re-point on failover — always connect the app to the *cluster* endpoint, not a specific instance's endpoint.

4. Path B — Self-managed on EC2

4.1 Launch the instance (shared)

```
aws ec2 run-instances \
  --image-id <latest-al2023-ami-id> \
  --instance-type t3.small \
  --subnet-id "$SUBNET_A" \
  --security-group-ids "$DB_SG" \
  --iam-instance-profile Name=gate-access-db-role \
  --block-device-mappings 'DeviceName=/dev/xvda,Ebs=
{VolumeSize=30,VolumeType=gp3,Encrypted=true}' \
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=gate-access-db}]' \
  --no-associate-public-ip-address \
  --query 'Instances[0].InstanceId'
```

> No public IP, and reach it via **AWS Systems Manager Session Manager** instead of opening SSH (port 22) to the internet: `aws ssm start-session --target <instance-id>`. Attach the `AmazonSSMManagedInstanceCore` policy to the instance role.

4.2 POSTGRES Install and configure

```
sudo dnf install -y postgresql16-server postgresql16
sudo /usr/bin/postgresql-16-setup --initdb
sudo systemctl enable --now postgresql-16

sudo -u postgres psql -c "CREATE DATABASE gate_access;"
sudo -u postgres psql -c "CREATE USER gate_app WITH ENCRYPTED PASSWORD '<strong-
password>';"
sudo -u postgres psql -c "GRANT ALL PRIVILEGES ON DATABASE gate_access TO gate_app;"
```

```
# /var/lib/pgsql/16/data/postgresql.conf
listen_addresses = '10.0.0.0/8'      # your VPC CIDR, not '*'
ssl = on

# /var/lib/pgsql/16/data/pg_hba.conf — replace the default trust/ident lines with:
hostssl gate_access gate_app 10.0.0.0/16 scram-sha-256

sudo systemctl restart postgresql-16
```

4.3 MYSQL Install and configure

```
sudo dnf install -y mysql-community-server
sudo systemctl enable --now mysqld

# grab the temporary root password, then run the hardening wizard
sudo grep 'temporary password' /var/log/mysqld.log
sudo mysql_secure_installation

sudo mysql -u root -p -e "
  CREATE DATABASE gate_access;
  CREATE USER 'gate_app'@'10.0.0.0/16' IDENTIFIED BY '<strong-password>';
  GRANT ALL PRIVILEGES ON gate_access.* TO 'gate_app'@'10.0.0.0/16';
  FLUSH PRIVILEGES;
"
```

```
# /etc/my.cnf.d/mysql-server.cnf
[mysqld]
bind-address = 0.0.0.0        # fine — the security group restricts source IPs
require_secure_transport = ON

sudo systemctl restart mysqld
```

4.4 Firewall (shared)

```
# only allow the app tier's security group, on the OS firewall too (defense in depth)
sudo firewall-cmd --permanent --add-rich-rule='rule family="ipv4" source
address="10.0.0.0/16" port protocol="tcp" port="5432" accept'   # or 3306 for MySQL
sudo firewall-cmd --reload
```

4.5 Point the app at it

```
export DATABASE_URL="postgresql+psycopg2://gate_app:<password>@<ec2-private-
ip>:5432/gate_access?sslmode=require"
pipenv run alembic upgrade head

export MYSQL_DATABASE_URL="mysql+pymysql://gate_app:<password>@<ec2-private-
ip>:3306/gate_access"
python3 -c "
from app_mysql.db.database import Base, engine
from app_mysql.db import models
Base.metadata.create_all(engine)
"
```

4.6 Backups (your responsibility on this path)

```
# Postgres — nightly dump to S3
pg_dump -Fc -h localhost -U gate_app gate_access > /tmp/gate_access.dump
aws s3 cp /tmp/gate_access.dump s3://<your-backup-bucket>/postgres/$(date +%F).dump

# MySQL — nightly dump to S3
mysqldump -h localhost -u gate_app -p gate_access > /tmp/gate_access.sql
aws s3 cp /tmp/gate_access.sql s3://<your-backup-bucket>/mysql/$(date +%F).sql
```

Put either command in a systemd timer or cron job, and separately schedule EBS snapshots (`aws dlm` lifecycle policy) for full-volume recovery.

5. Migrating existing data

Your current Postgres database (`itc@54.246.247.31/gate_access`) presumably already has real rows. Moving it to a new Postgres host (Aurora or EC2) is a straight dump/restore; there is no MySQL data yet to migrate since `app_mysql` is a fresh schema.

```
# from a host that can reach both the old box and the new one
pg_dump -h 54.246.247.31 -U itc -Fc gate_access > gate_access.dump
pg_restore -h <new-host> -U gate_app -d gate_access --no-owner --no-privileges
gate_access.dump
```

> If you later want the *existing* Postgres data available in MySQL too, that needs a row-level ETL (types don't map 1:1 — e.g. `JSONB` → `JSON` , unbounded `VARCHAR` → sized `VARCHAR` / `TEXT`) rather than a dump/restore. AWS Database Migration Service (DMS) can do this with a Postgres source and MySQL target endpoint if/when you need it — worth a separate pass, not a one-line command.

6. Production hardening checklist

- Database has **no public IP** and is not in a public subnet (both paths above already do this).
- Security group allows only the app tier's security group/IP range, on the DB port only.
- TLS enforced on the connection (`sslmode=require` for Postgres, `require_secure_transport=ON` for MySQL) — download the RDS CA bundle for Aurora clients: `curl -o rds-ca.pem https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem`.
- Storage encrypted at rest (`--storage-encrypted` on Aurora, encrypted EBS volume on EC2).
- Automated backups with a real retention period — 7+ days at minimum; verify a restore at least once.
- Credentials in Secrets Manager / Parameter Store, not in `.env` or source (§2.3).
- A distinct, least-privilege application DB user (`gate_app`) — never connect the app as the master/root user day-to-day.
- CloudWatch alarms on CPU, storage, and connection count; for Aurora, enable Enhanced Monitoring.
- If Aurora: a reader instance in a second AZ, so failover is automatic rather than a page-in-the-night rebuild.

7. Rotate the credentials already exposed in this repo

Flagged earlier in this project and still outstanding — do this regardless of which hosting path you pick, since both involve creating a *new* user/password anyway:

- `app/db/database.py` has a live Postgres password hardcoded in source (`itc:082001@54.246.247.31`). Once you cut over to the new host, rotate that password on the old server too, and remove the hardcoded fallback per §2.1.
- `.env` contains a plaintext OpenAI API key. Rotate it at `platform.openai.com` and load the replacement from Secrets Manager in production, not from a committed/shared file.

Appendix A. MySQL from Lambda — driver, Aurora, IAM auth

Applies when the Postgres-calling Lambda functions are moved over to `app_mysql`. Covers which MySQL driver to package, how it behaves against Aurora specifically, and how to avoid static DB passwords entirely by using IAM authentication from the function's execution role.

A.1 Driver choice

Driver	Lambda packaging	Notes
PyMySQL RECOMMENDED	Pure Python — `pip install` straight into the layer/zip, no compilation, identical artifact for x86_64 and arm64 Lambda.	Already the driver used in `app_mysql/db/database.py` (`mysql+pymysql://`) — one driver, one set of quirks, across the app and the Lambdas. Supports TLS and IAM auth tokens natively.
mysql-connector-python	Pure-Python mode packages fine; no real advantage over PyMySQL to offset the larger package size.	Historically more SQLAlchemy-version compatibility friction. Its C-accelerated mode has the same build problem as mysqlclient below.
mysqlclient	C extension linked against `libmysqlclient` — must be compiled inside the Lambda runtime's OS (Amazon Linux 2023), via `sam build --use-container` /a matching Docker image, or a community-maintained prebuilt layer.	Fastest for high-throughput bulk row fetches, but that's rarely the bottleneck in a request/response Lambda — cold start and the TLS handshake to Aurora dominate instead. Only reach for this after profiling shows the driver itself is the bottleneck.

A.2 Packaging PyMySQL as a Lambda layer

```
mkdir -p layer/python
pip install pymysql -t layer/python
cd layer && zip -r ../pymysql-layer.zip python && cd ..

aws lambda publish-layer-version \
  --layer-name pymysql \
  --zip-file fileb://pymysql-layer.zip \
  --compatible-runtimes python3.12 \
  --compatible-architectures x86_64 arm64

# attach to a function:
aws lambda update-function-configuration \
  --function-name <your-lambda> \
  --layers <layer-arn-from-above>
```

A.3 Connecting to Aurora MySQL over TLS

Aurora expects encrypted connections. Bundle the RDS CA certificate in the same layer (or the deployment package) and reference it in `connect_args`:

```
curl -o rds-ca-bundle.pem https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem
# ship this file in the layer, e.g. layer/python/rds-ca-bundle.pem -> /opt/python/rds-ca-bundle.pem at runtime
```

```
from sqlalchemy import create_engine
from sqlalchemy.pool import NullPool

engine = create_engine(
    DATABASE_URL,   # mysql+pymysql://gate_app:<password>@<aurora-cluster-endpoint>:3306/gate_access
    connect_args={"ssl": {"ca": "/opt/python/rds-ca-bundle.pem"}},
    poolclass=NullPool,   # see A.5 — don't let a frozen Lambda container hold a stale pooled connection
)
```

A.4 IAM database authentication (no stored password at all)

Since a Lambda function already has an execution role, prefer generating a short-lived auth token over storing a DB password in Secrets Manager. PyMySQL accepts the token as the connection password.

1. Enable IAM auth on the cluster:

```
aws rds modify-db-cluster \
  --db-cluster-identifier gate-access-mysql \
  --enable-iam-database-authentication \
  --apply-immediately
```

2. Create/alter the DB user to use the IAM auth plugin (run against the cluster as the master user):

```
CREATE USER 'gate_app_iam'@'%' IDENTIFIED WITH AWSAuthenticationPlugin AS 'RDS';
GRANT SELECT, INSERT, UPDATE, DELETE ON gate_access.* TO 'gate_app_iam'@'%';
FLUSH PRIVILEGES;
```

3. Grant the Lambda's execution role permission to mint a connect token, scoped to that one DB user:

```
{
  "Effect": "Allow",
  "Action": "rds-db:connect",
  "Resource": "arn:aws:rds-db:<region>:<account-id>:dbuser:<cluster-resource-
id>/gate_app_iam"
}
```

`<cluster-resource-id>` comes from `aws rds describe-db-clusters --db-cluster-identifier gate-access-mysql --query 'DBClusters[0].DbClusterResourceId'`.

4 Generate the token and connect, inside the Lambda handler:

```
import boto3
from sqlalchemy import create_engine
from sqlalchemy.pool import NullPool

rds = boto3.client("rds")

def _build_engine():
    token = rds.generate_db_auth_token(
        DBHostname="<aurora-cluster-endpoint>",
        Port=3306,
        DBUsername="gate_app_iam",
    )
    url = "mysql+pymysql://gate_app_iam:@<aurora-cluster-endpoint>:3306/gate_access"
    return create_engine(
        url,
        connect_args={
            "password": token,
            "ssl": {"ca": "/opt/python/rds-ca-bundle.pem"},
        },
        poolclass=NullPool,
    )
```

> Tokens are valid for 15 minutes — generate a fresh one per cold start / per invocation rather than caching it across warm invocations for longer than that window.

A.5 Connection pooling from Lambda — use RDS Proxy

Lambda's concurrency model means many containers can each open a connection simultaneously; at any real traffic level this exhausts Aurora's max connections. Put RDS Proxy in front and let *it* pool, rather than pooling in the function:

```
aws rds create-db-proxy \
  --db-proxy-name gate-access-mysql-proxy \
  --engine-family MYSQL \
  --auth '[{"AuthScheme":"SECRETS","IAMAuth":"REQUIRED","SecretArn":"<secret-arn-if-not-using-iam-auth-above>"}]' \
  --role-arn <proxy-iam-role-arn> \
  --vpc-subnet-ids "$SUBNET_A" "$SUBNET_B" \
  --vpc-security-group-ids "$DB_SG"
```

Point `DATABASE_URL` /the Lambda's connection host at the proxy endpoint instead of the cluster endpoint, keep `poolclass=NullPool` in SQLAlchemy (the proxy is the pool now), and RDS Proxy also supports IAM auth end-to-end so §A.4's token approach still applies.